Functorial Composition over Task Decomposition: A Categorical Kernel for AI-Human Computation

Anonymous Author
mo:os Project
author@example.com

Multi-agent AI frameworks decompose tasks top-down, producing orchestrations that lack formal structural guarantees. We present `mo:os`, a runtime kernel realized as a symmetric monoidal category where computational entities are objects and state transitions are exactly four natural transformations (ADD, LINK, MUTATE, UNLINK) over a typed graph. Applying Lawvere's functorial semantics, `mo:os` separates syntax (a JSON-schema-validated ontology of 21 object kinds and 16 morphism types) from semantics (a deterministic Go execution engine implementing a catamorphism $state(t) = fold(log[0..t])$). By restricting LLMs to emit typed morphisms rather than orchestration plans, complex behavior emerges from bottom-up operadic composition rather than unstructured decomposition. We formalize the Container Category, prove closure under parallel and sequential wiring, present the Recursive Semantic Bridge as a structure-preserving functor, and report implementation metrics from a working kernel: 3,729 lines of Go, 65 nodes, 80 wires, 14 HTTP endpoints, and deterministic replay from an append-only morphism log.

1 Introduction

The rapid adoption of large language models (LLMs) as orchestrators of multi-agent systems has exposed a fundamental architectural tension: the frameworks that decompose tasks—LangChain, CrewAI, AutoGen, and their successors—rely on the LLM's embedding space to simultaneously serve as computation engine, state router, and correctness oracle. This conflation produces what we term the *semantic bottleneck*: as task complexity grows, recombination of sub-results depends on ad-hoc aggregation that is brittle to hallucination and context-window overflow.

We propose an inversion of control. Rather than asking the LLM to *orchestrate*, we restrict it to the role of a *Fuzzy Processing Unit* (FPU) that emits candidate morphisms over a strictly typed categorical state machine. The deterministic kernel accepts or rejects these morphisms against a semantic registry before committing them to an append-only log. This separation—syntax from semantics, in the sense of Lawvere [8]—yields structural guarantees that top-down decomposition cannot provide.

`mo:os` is an open-source (MIT) runtime kernel, implemented in Go, that realizes this architecture. Its categorical foundation draws on Fong and Spivak's compositional framework [5], Spivak's operadic wiring diagrams [14, 12], and Yau's finite presentation results for wiring diagram operads [15]. Unlike Catlab (scientific computing) or Statebox (blockchain consensus) [3], `mo:os` targets the operational gap between LLM inference and industrial graph state management.

Contributions.

1. We define the *Container Category* $\mathscr{C}$ and prove it is closed under parallel and sequential wiring (Section 3).

Submitted to:
ACT 2026

© A. Author
This work is licensed under the
Creative Commons Attribution License.

2. We formalize four invariant natural transformations and the operadic validation algebra that constrains them (Section 4).
3. We construct the *Recursive Semantic Bridge* as a structure-preserving functor $F\colon \mathbf{Syn} \to \mathbf{Sem}$ and show that hydration stages correspond to strata of realization (Section 5).
4. We present *System 3* DAG reasoning as an extension of LogicGraph [1] (Section 6).
5. We report metrics from a working implementation: 3,729 lines of Go, deterministic catamorphic replay, and a 21-kind ontology validated by an operad registry (Section 7).

2 Background: Functorial Semantics and Operadic Composition

2.1 Lawvere's Functorial Semantics

Lawvere [8] established that an algebraic theory $\mathbf{T}$ can be separated from its models by defining models as product-preserving functors $M\colon \mathbf{T} \to \mathbf{Set}$. This separation of syntax (the theory) from semantics (the model) is the conceptual foundation of `mo:os`: the JSON ontology is our syntax category, and the Go kernel is a model that evaluates it.

2.2 Operadic Wiring Diagrams

Spivak [14] formalized wiring diagrams as an operad $\mathscr{W}$ where operations describe how boxes with typed ports are interconnected. Rupel and Spivak [12] extended this to temporal wiring diagrams with directed, length-bearing wires suitable for modeling dynamic information flow. Yau [15] proved that $\mathscr{W}$ admits a finite presentation with 8 generators and 28 relations.

In `mo:os`, the 16 morphism types of our ontology correspond to generators of a sub-operad of $\mathscr{W}$, and the semantic registry enforces the operadic relations: each wire must connect admissible port types with correct directionality.

2.3 Dynamic Operads and Agent Composition

Shapiro et al. [13] introduced *dynamic operads* via the monoidal double category **Org**, showing how organized systems adapt their operadic structure in response to internal and external pressures. This framework supports the view of `mo:os` as a dynamic categorical system where the graph topology evolves through morphism application while preserving algebraic invariants.

Bonchi, Pavlovic, and Sobocinski [4] extended functorial semantics from algebraic to relational theories, providing tools for reasoning about agent interactions as structure-preserving maps between relational networks—a pattern we exploit in the `mo:os` wiring algebra.

3 The Container Category

Definition 1 (Container Category)**.** *The* Container Category $\mathscr{C}$ *has:*

- ***Objects****: typed nodes* $n = (urn, \tau, \sigma, \pi)$ *where urn is a unique identifier,* $\tau \in \mathscr{T}$ *is a type drawn from the ontology (*$|\mathscr{T}| = 21$*),* $\sigma \in \{S_0, \ldots, S_4\}$ *is a stratum, and* π *is a payload.*
- ***Morphisms****: directed wires* $w\colon n_1 \to n_2$ *specified by the 4-tuple* (src, p_s, tgt, p_t) *where* p_s, p_t *are typed ports.*

- ***Identity***: *for each node n, the identity wire* id_n *is the trivial self-loop (elided in practice).*
- ***Composition***: *wires compose when the target port of one matches the source port of the next, subject to the operadic constraints of the semantic registry.*

The graph state at time t is the carrier of the catamorphism:

$$state(t) = \mathrm{fold}\big(log[0..t],\ \varnothing,\ eval\big) \tag{1}$$

where $\varnothing$ is the empty graph (initial object) and *eval* applies a single morphism envelope to produce a new state.

Definition 2 (Stratum Lattice)**.** *The five strata form a partial order:*

$$S_0\ (Authored) \leq S_1\ (Validated) \leq S_2\ (Materialized) \leq S_3\ (Evaluated) \leq S_4\ (Projected)$$

Higher strata may depend on lower strata; lower strata must not depend on higher strata for meaning. S_4 *projections are never ground truth.*

Proposition 1 (Closure under Wiring)**.** *Let* G_1, G_2 *be valid graph states in* $\mathscr{C}$*. The parallel composition* $G_1 \otimes G_2$ *(disjoint union of nodes and wires) and the sequential composition* $G_1 \circ G_2$ *(wire matching on shared ports) both yield valid graph states in* $\mathscr{C}$*, provided all wires satisfy the operadic constraints of the semantic registry.*

Proof sketch. Parallel composition preserves validity because disjoint union introduces no new port conflicts. Sequential composition requires that matched ports have compatible types and directions, which is enforced by the registry's `ValidateLink` function checking source port direction ("out"), target port direction ("in"), and target type membership in the source port's admissible target set. □

3.1 Ownership as Full Subcategory

The OWNS morphism (*MOR01*) induces a forest structure on $\mathscr{C}$. For any actor a, the *scope* of a is the full subcategory $\mathscr{C}_a \hookrightarrow \mathscr{C}$ consisting of all nodes transitively reachable from a via OWNS wires. This is computed by breadth-first search under a read lock, yielding all internal wires—those whose source and target both lie in $\mathscr{C}_a$.

4 The Semantic Bottleneck and Invariant Morphisms

4.1 The Semantic Bottleneck

The prevailing paradigm for multi-agent orchestration relies on the LLM's embedding space as both computation engine and state router. We formalize this structural failure as the *semantic bottleneck*: a system architecture where semantic inference and operational state management share a single, untyped channel. As task complexity grows, this channel saturates, producing hallucinated execution paths and brittle error recovery.

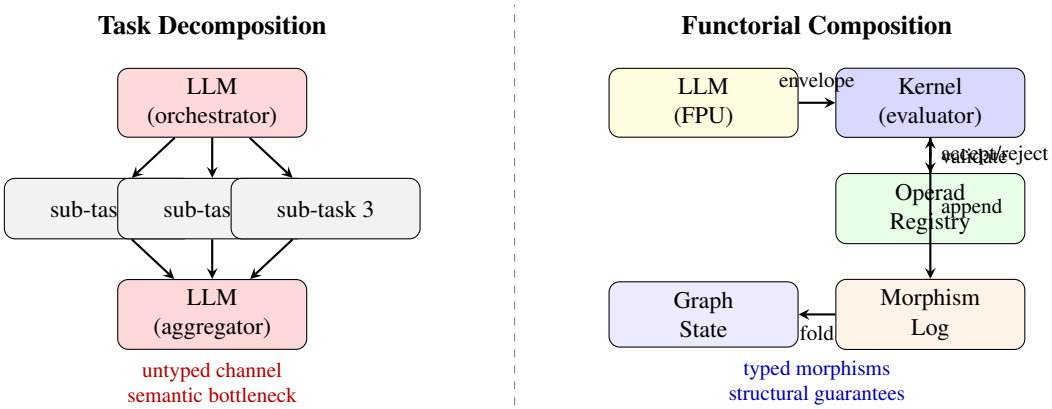

Figure 1: The semantic bottleneck (left): LLMs serve as both orchestrator and aggregator through an untyped channel, producing brittle compositions. Functorial composition (right): the LLM acts as a Fuzzy Processing Unit emitting typed morphism envelopes, validated by the operad registry before commitment to the append-only log.

4.2 Four Invariant Natural Transformations

`mo:os` resolves the bottleneck by restricting all state mutations to exactly four natural transformations:

NT	Signature	Semantic	Guard
ADD	$\varnothing \to \text{Node}$	Create node	Duplicate URN $\Rightarrow$ `ErrNodeExists`
LINK	$N \times N \to \text{Wire}$	Create wire	Operadic port validation
MUTATE	$N \to N$	Update payload	Version CAS + S_0 block
UNLINK	$\text{Wire} \to \varnothing$	Remove wire	Wire not found $\Rightarrow$ error

These four operations are *complete*: any graph state reachable from the empty graph can be expressed as a finite composition of ADD, LINK, MUTATE, and UNLINK envelopes. The LLM (as FPU) proposes candidate envelopes; the kernel validates and commits them.

4.3 Operadic Validation

The semantic registry $\mathscr{R}$ is an operad algebra over the port structure of $\mathscr{C}$. For each type $\tau \in \mathscr{T}$, the registry specifies:

- **Allowed strata**: the set of valid strata for instances of τ.
- **Port specifications**: for each named port, its direction (in/out) and admissible target types.
- **Mutability**: whether instances admit MUTATE.

A LINK envelope $w\colon n_1 \xrightarrow{(p_s,p_t)} n_2$ is valid if and only if: (1) p_s exists on $\tau(n_1)$ with direction "out", (2) p_t exists on $\tau(n_2)$ with direction "in", and (3) $(\tau(n_2), p_t)$ is in the admissible target set of p_s. This three-part check implements the operadic composition constraints of $\mathscr{W}$.

$$S_0 \xrightarrow{\text{schema}} S_1 \xrightarrow{\text{Materialize}} S_2 \xrightarrow{\text{ApplyProgram}} S_3 \xrightarrow{\text{Scope}} S_4$$

JSON files	validated	programs	graph state	projections
Authored	Validated	Materialized	Evaluated	Projected

Figure 2: The hydration pipeline as stratum promotion. Each stage corresponds to a kernel operation: schema validation ($S_0 \to S_1$), materialization into Programs ($S_1 \to S_2$), fold evaluation ($S_2 \to S_3$), and scoped projection via BFS ($S_3 \to S_4$). S_4 outputs are never ground truth.

5 The Recursive Semantic Bridge

We formalize the `mo:os` architecture as a realization of Lawvere's functorial semantics. The database graph serves as the abstract syntax tree (AST) of our language.

Definition 3 (Syntax Category)**.** **Syn** *has objects = Container types (the 21 kinds of the ontology) and morphisms = wiring diagrams (the 16 morphism types).*

Definition 4 (Semantics Category)**.** **Sem** *has objects = Go runtime states and morphisms = deterministic kernel evaluations (the fold step of Equation 1).*

The *Recursive Semantic Bridge* is the structure-preserving functor:

$$F : \mathbf{Syn} \to \mathbf{Sem} \tag{2}$$

$$\begin{tikzcd} \mathbf{Syn} \arrow[r, "F"] \arrow[d, "\alpha"'] & \mathbf{Sem} \arrow[d, "F(\alpha)"] \\ \mathbf{Syn}' \arrow[r, "F"'] & \mathbf{Sem}' \end{tikzcd} \tag{3}$$

where α represents a morphism (state transition) in the syntax, and $F(\alpha)$ is the corresponding Go kernel evaluation. The functor preserves composition: applying two morphisms sequentially in **Syn** yields the same result as applying their F-images sequentially in **Sem**.

5.1 Hydration as Stratum Promotion

The hydration pipeline maps directly onto the stratum lattice:

Stage	Stratum	Kernel Operation
Authored	S_0	JSON instance files on disk
Validated	S_1	Schema check (JSON Schema draft-07)
Materialized	S_2	`Materialize`: instance $\to$ Program
Evaluated	S_3	`ApplyProgram`: fold into graph state
Projected	S_4	`ScopedSubgraph`: BFS lens

The key invariant is that S_4 projections are *never* ground truth: they are views derived from the evaluated state, and the graph can always be reconstructed from the morphism log via replay (Equation 1).

5.2 Harness Agnosticism

Because the functor F maps syntax to semantics uniformly, any external tool—an LLM, a database, a UI framework—is merely an object in **Syn** whose interaction with the graph is governed by F. This achieves *harness agnosticism*: the UI surface is a direct, consistent projection of the categorical state, requiring no intermediary state-management synchronization. Nested sub-workspaces are submodules in the categorical composition, inheriting structural integrity via algebraic closure.

6 System 3: Multi-Path DAG Reasoning

We extend LogicGraph [1] to define System 3 reasoning over the morphism space:

- **System 1**: Intuitive token prediction (standard LLM inference).
- **System 2**: Linear chain-of-thought (sequential reasoning).
- **System 3**: DAG exploration over typed morphism space.

In System 3, the deterministic kernel spawns candidate sub-graphs where each edge is a structure-preserving morphism validated against the operad registry. By applying tree search heuristics (MCTS or beam search) over the morphism space, `mo:os` simulates branching future states. Because each branch is restricted to valid categorical syntax, the search space is automatically pruned of structurally inconsistent outcomes.

This enables high-stakes reasoning—infrastructure deployment, data migration, multi-step planning—where the error surface of a single LLM prompt would be prohibitively large. The typed morphism space provides a natural pruning function: invalid compositions are rejected before evaluation, and the catamorphic replay guarantees that any accepted path can be audited.

7 Implementation

`mo:os` is implemented in Go (1.22+) as a single-binary kernel with zero external dependencies beyond the standard library. The architecture separates pure computation from effectful state management:

Layer	Package	Responsibility
Types	`cat`	Core types: Node, Wire, GraphState, Envelope, Program. 4 morphism types, 5 strata.
Pure fold	`fold`	Catamorphism: `Evaluate`, `Replay`. Zero IO imports.
Operad	`operad`	Semantic registry: 21 TypeSpecs with port/direction/target validation.
Effect shell	`shell`	`Runtime`: RWMutex-guarded state, `Apply`, `SeedIfAbsent`.
Transport	`transport`	14 HTTP routes on port 8000.
Hydration	`hydration`	`Materialize`: instance JSON → Program. `HydrateAll`: batch boot.

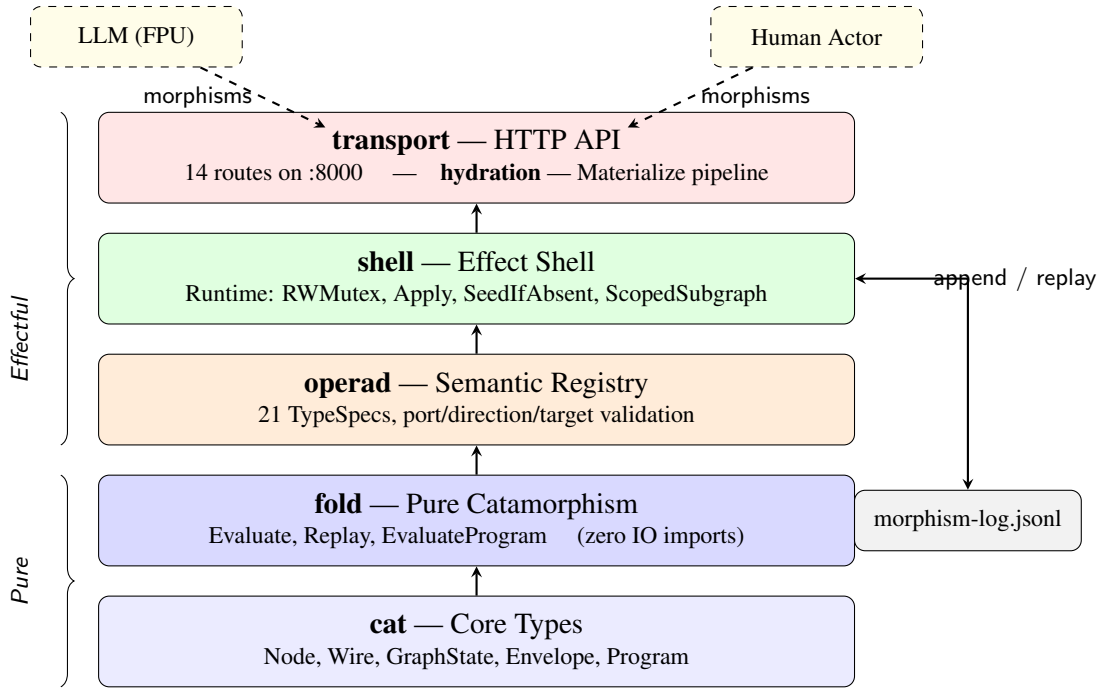

Figure 3: Layered architecture of the `mo:os` kernel. The pure core (`cat` + `fold`) contains zero IO imports. The effect shell wraps it with `sync.RWMutex` for concurrent access. Both LLM agents and human actors interact exclusively through typed morphism envelopes.

7.1 Catamorphic Replay

The morphism log is an append-only JSONL file. State reconstruction is a left fold:

Listing 1: Pure replay (simplified)

```
func Replay(log []Envelope) GraphState {
    state := NewGraphState() // empty
    for _, env := range log {
        state = Evaluate(state, env)
    }
    return state
}
```

The `fold` package imports no IO packages (`os`, `net`, `sync`); it operates purely on values. The effect shell (`shell.Runtime`) wraps the pure fold with `sync.RWMutex` for concurrent read access and exclusive write access.

7.2 Idempotent Seeding

Boot uses `SeedIfAbsent`, which absorbs `ErrNodeExists` and `ErrWireExists`. This makes repeated boots idempotent: the first boot creates 13 seed morphisms; subsequent boots replay the log and produce zero new entries.

7.3 Metrics

Metric	Value
Go source lines	3,729
Ontology object kinds	21
Ontology morphism types	16
Invariant natural transformations	4
Strata	5
HTTP routes	14
Nodes (post-hydration)	65
Wires (post-hydration)	80
Morphism log depth	145
Test packages (all green)	6
External dependencies	0

8 Related Work

Categorical frameworks. Catlab [11] provides Julia libraries for computational category theory; Statebox [3] applies Petri nets to blockchain consensus; Lynch and Breiner [9] implement nested wiring diagrams in Semagrams. None target the operational gap between LLM inference and runtime graph state management.

Multi-agent orchestration. LangChain, CrewAI, AutoGen, and LangGraph rely on LLM-driven task decomposition. These frameworks lack formal structural guarantees: correctness depends on the LLM maintaining coherent state across unbounded context windows.

Operadic composition. Spivak's wiring diagram operads [14, 12] and Yau's finite presentation results [15] provide the mathematical foundation for our port-typed composition. Shapiro et al.'s dynamic operads [13] formalize how operadic structure evolves—a pattern `mo:os` realizes through its mutable graph topology.

Functorial semantics for computation. Bonchi et al. [4] extend functorial semantics to relational theories. Gu and Zanasi [7] apply functorial semantics as a unifying framework for logic programming. `mo:os` applies these ideas to AI-human computation, where the "programs" are morphism envelopes emitted by both human actors and LLM agents.

Category theory for AI. Pan [10] proposes a categorical framework at the token level of transformer architectures. Gavranović et al. [6] develop an algebraic theory of deep learning architectures using 2-categories of parametric maps. These works formalize AI *internals* categorically; `mo:os` instead formalizes the *interface* between AI agents and shared state.

Neuro-symbolic reasoning. LogicGraph [1] benchmarks multi-path reasoning in high-dimensional search spaces. `mo:os`'s System 3 extends this by embedding DAG reasoning within a categorically typed morphism space, providing structural pruning guarantees absent in untyped search.

9 Conclusion

We have demonstrated that the transition from top-down task decomposition to bottom-up functorial composition provides a rigorous foundation for AI-human computation. By anchoring LLMs as coprocessors within a typed categorical kernel, `mo:os` resolves the semantic bottleneck while preserving the expressiveness needed for complex multi-agent workflows. The working implementation—3,729 lines of Go with zero dependencies, 21 object kinds validated by an operad registry, and deterministic catamorphic replay—shows that mathematical rigor is not an obstacle to practical systems but rather their most robust enabling technology.

Future work includes: (1) an MCP bridge [2] exposing the kernel as a Model Context Protocol server with SSE transport; (2) a benchmark functor implementing the classifying functor $\mathscr{T} \to \mathbf{Met}$ from provider types to a metric space of benchmark scores; and (3) quantitative comparison of morphism-based error recovery rates against traditional LLM orchestration frameworks.

References

[1] Anonymous (2025): *Benchmarking Multi-Path Logical Reasoning in High-Dimensional Search Spaces*. *Internal Report.*

[2] Anthropic (2024): *Model Context Protocol (MCP)*. `https://modelcontextprotocol.io`.

[3] Jelle Herold Boer et al. (2019): *Mathematical Specification of the Statebox Language*. In: *Proceedings of ACT 2019.*

[4] Filippo Bonchi, Dusko Pavlovic & Pawel Sobocinski (2017): *Functorial Semantics for Relational Theories*. *arXiv preprint arXiv:1711.08699.*

[5] Brendan Fong & David I Spivak (2018): *Seven Sketches in Compositionality: An Invitation to Applied Category Theory*. Cambridge University Press.

[6] Bruno Gavranović, Paul Lessard, Andrew Duber & Daniel Murfet (2024): *Categorical Deep Learning: An Algebraic Theory of Architectures*. *arXiv preprint arXiv:2402.15332.*

[7] Tao Gu & Fabio Zanasi (2021): *Functorial Semantics as a Unifying Perspective on Logic Programming*. In: *9th Conference on Algebra and Coalgebra in Computer Science (CALCO 2021)*, Schloss Dagstuhl – Leibniz-Zentrum für Informatik, pp. 17:1–17:22.

[8] F. William Lawvere (1963): *Functorial Semantics of Algebraic Theories*. *Proceedings of the National Academy of Sciences* 50(5), pp. 869–872.

[9] Owen Lynch & Spencer Breiner (2023): *Nested wiring diagrams in Semagrams*. In: *Proceedings of ACT 2023*, *EPTCS* 397.

[10] Wuming Pan (2024): *Token Space: A Category Theory Framework for AI Computations*. *arXiv preprint arXiv:2404.11624.*

[11] Evan Patterson, Andrew Baas & James Fairbanks (2024): *Catlab.jl: A Framework for Applied Category Theory in Julia*. `https://github.com/AlgebraicJulia/Catlab.jl`.

[12] Dylan Rupel & David I Spivak (2013): *The operad of temporal wiring diagrams: formalizing a graphical language for discrete-time processes*. *arXiv preprint arXiv:1307.6894.*

[13] Brandon T Shapiro & David I Spivak (2022): *Dynamic Operads, Dynamic Categories: From Deep Learning to Prediction Markets*. *arXiv preprint arXiv:2205.03906.*

[14] David I Spivak (2013): *The operad of wiring diagrams: formalizing a graphical language for databases, plug-ins, and concurrent routing*. *arXiv preprint arXiv:1305.0297.*

[15] Donald Yau (2018): *Operads of Wiring Diagrams*. *Lecture Notes in Mathematics* 2192, Springer. ArXiv:1512.01602.